FitPal — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and the two memories

Keep the two memories separate — that's the whole design:

Memory	Keyed by	Holds	Pieces
checkpointer (`SqliteSaver`)	`thread_id`	this conversation's messages	the graph's `compile(checkpointer=...)`
Store (`InMemoryStore`)	`user_id`	durable facts (goal, gear, injury)	`src/memory/store.py`

Files: `src/state.py` (`ChatState`: messages+add_messages, user_id, recalled), `src/memory/store.py` (recall/save/forget), `src/memory/writer.py` (decide), `src/nodes.py` (recall/chat/write_back), `src/graph.py` (`recall → chat → write_back`). Build order: a checkpointer-only chat (watch a new thread forget you) → Store + recall → write-back decision → `forget_me`.

Hint 2 — the Store, keyed by user, with stable keys

```
def namespace_for(user_id):            # keyed by USER, not thread
    return ("fitness", user_id)
def recall_facts(store, user_id):      # -> list of fact strings
    return [i.value["fact"] for i in store.search(namespace_for(user_id))]
def save_fact(store, user_id, key, fact):
    store.put(namespace_for(user_id), key, {"fact": fact})
def forget_user(store, user_id):       # delete EVERY item, not one key
    ns = namespace_for(user_id)
    for item in store.search(ns):
        store.delete(ns, item.key)
```

(Write the comprehensions as plain loops.) Two things matter: the namespace is `user_id` (so a new `thread_id` still recalls you), and facts are saved under **stable keys** (`goal`, `injury`,

`equipment`) so telling FitPal a new goal **updates** the old one instead of stacking a contradiction.

Hint 3 — the nodes, and how each gets what it needs

```
recall_node(state, *, store):        # store injected because it's a parameter
    return {"recalled": "; ".join(recall_facts(store, state["user_id"]))}
make_chat_node(model) -> chat_node(state):   # model captured by the closure
    system = chat prompt with {recalled}
    model.invoke([SystemMessage] + messages)
make_write_back_node(model) -> write_back_node(state, *, store):   # both!
    last = last HumanMessage; d = decide_memory(model, last.content)
    if d.should_remember: save_fact(store, state["user_id"], d.key, d.fact)
graph: START -> recall -> chat -> write_back -> END
       compile(checkpointer, store)
```

The subtlety: `recall_node` needs only the store (inject it via a `store` parameter), `chat_node` needs only the model (capture it in a closure), and `write_back_node` needs **both** — closure for the model, `store` parameter for injection. Compile with `store=...` so the injection works.

Hint 4 — deciding what to keep, and forgetting

`decide_memory` is `prompt | model.with_structured_output(MemoryDecision)` where `MemoryDecision` is `{should_remember, key, fact}`. The prompt does the judgement: durable fitness facts (goal, equipment, injury, schedule) yes; "felt tired today" no; reuse stable keys. Most messages should save **nothing** — a write-back that fires every turn is a junk drawer.

For `forget_me`: `forget_user` must `search` the namespace and delete **every** item (health data — a partial forget is a broken promise). Prove it the honest way: save two facts, `forget_user`, then confirm `recall_facts` is empty **and** a brand-new session recalls nothing.

Still stuck? The whole project, every file, is in the complete solution.